ALGOINVEST&TRADE

OPTIMISATION D'UNE STRATÉGIE D'INVESTISSEMENT AVEC DES ALGORITHMES PYTHON

I - Introduction
a. Le contexte 2
b. Le problème du sac à dos

II - Solution brute-force
a. L'algorithme 4
b. Limites et transition vers une solution plus optimisé

III - Solution optimisé
a. La programation dynamique 6
b. L'algorithme dynamique en pseudocode

IV - Comparaison d'efficacité et de performances, avec Big-O
a. Définition notation Big-O 13
b. Brute-force vs version optimisé (programmation dynamique)

V - Algorithme vs humain : Comparaison sur des données antérieurs 17
a. Présentation des données
b. Rapports d'exploration

I - INTRODUCTION

a. Le contexte

- **Contexte :** La société d'investissement AlgoInvest&Trade souhaitent optimiser leur stratégie d'investissement à l'aide d'algorithmes afin de maximiser le profit de ses clients. Une stratégie d'investissement est une sélections d'actions choisies parmi les actions disponibles, dont le coût total respecte le budget imparti.

- **Objectif** : Créer un algorithme qui renvoie la sélection d'actions la plus rentable après deux ans d'investissement, sous le budget imparti.

- **Contraintes :**
  - Chaque action ne peut être acheté qu'une fois
  - L'action ne peut pas être acheté en fraction
  - Le budget est de 500 euros par client

- **Chaque action possède :**
  - un nom
  - un coût
  - un bénéfice (en %)

I - INTRODUCTION

b. Le problème du sac à dos

- **Définissons le problème :**

On se retrouve dans un problème classique en algorithmique : le problème du sac à dos (Knapsack problem) . C’est un problème d'optimisation combinatoire, qui prend en compte un contenant qui peut contenir un poids maximum (le budget de 500 euros dans notre cas), et une liste d’objets qui pèsent un poids et ont une valeur (la liste d’actions). Le but est de trouver de manière efficace la combinaison d’objets qui apportera le plus de valeur total, tout en respectant le poids maximum du contenant.

- **Comment résoudre ?**

L’approche de résolution la plus instinctive est celle qui consiste à essayer toutes les combinaisons d’actions, et à garder la séléction la plus rentable sous la contrainte budgetaire. C’est ce qu’on appelle : la solution brute-force .

II - SOLUTION BRUTE-FORCE

a. L'algorithme

- **Principe:** La solution brute-force essaie toutes les combinaisons d'actions possibles, et lorsque la combinaison respecte le budget, elle la compare avec la combinaison la plus rentable enregistré dans la boucle (initialisé à 0), et stocke la plus rentable d'entre les deux.

- **Algorithme en pseudocode :** Ci-dessous une représentation en pseudo-code de la fonction que j'ai élaboré pour optimiser nos stratégies d'investissement, en utilisant un algorithme brute-force. La fonction prend en paramètre une variable "liste_actions" qui est une liste d'instances de la classe Action.

```
FONCTION brute_force(liste_actions):
  meilleure_combinaison = {combinaison_actions = [ ], cout_total = 0, benefice_total = 0}
  POUR combinaison_possible (liste_actions) :
    cout_total = 0
    benefice_total = 0
    POUR action DANS combinaison_possible:
      cout_total += action.cout
      benefice_total += action.benefice_euros
    SI cout_total <= 500 ET benefice_total > meilleure_combinaison["benefice_total"] :
      meilleure_combinaison["combinaison_actions"] = combinaison_possible
      meilleure_combinaison["cout_total"] = cout_total
      meilleure_combinaison["benefice_total"] = benefice_total

  RETOURNE meilleure_combinaison
```

II - SOLUTION BRUTE-FORCE

b. Limites et transition vers une solution optimisé

- **Limites :**

L'algorithme est efficace pour une courte portion d'action, mais le temps d'exécution devient trop long lorsque le nombre d'actions augmente. Ce qui nous amène à réfléchir à une solution plus optimisé qui permettra de calculer un nombre important d'actions.

- **Comment optimiser ?**

Il existe differentes approche de résolution pour le problème du sac à dos, les plus utilisés sont :

- L'approche gloutonne : Avantage → Rapide
Inconvenient → Renvoie parfois une solution proche de la meilleure solution

- La programmation dynamique : Avantage → Précis
Inconvénient → Coûteux en ressources (temps & mémoire) mais beaucoup moins que la brute-force

Nous allons privilégier la programmation dynamique pour notre cas.

III - SOLUTION OPTIMISÉ

a. La programation dynamique

- **Qu’est ce que la programmation dynamique ?**

La programmation dynamique est une méthode utilisée pour résoudre des problèmes algorithmiques complexes en les décomposant en sous-problèmes plus petits. Elle est particulièrement utile lorsque ces sous-problèmes se chevauchent (c’est-à-dire que les mêmes sous-problèmes demandent d’être résolu plusieurs fois au cours de la résolution), et en stockant leur résultats afin d’éviter de les recalculer (et donc d’économiser de la puissance de calcul).

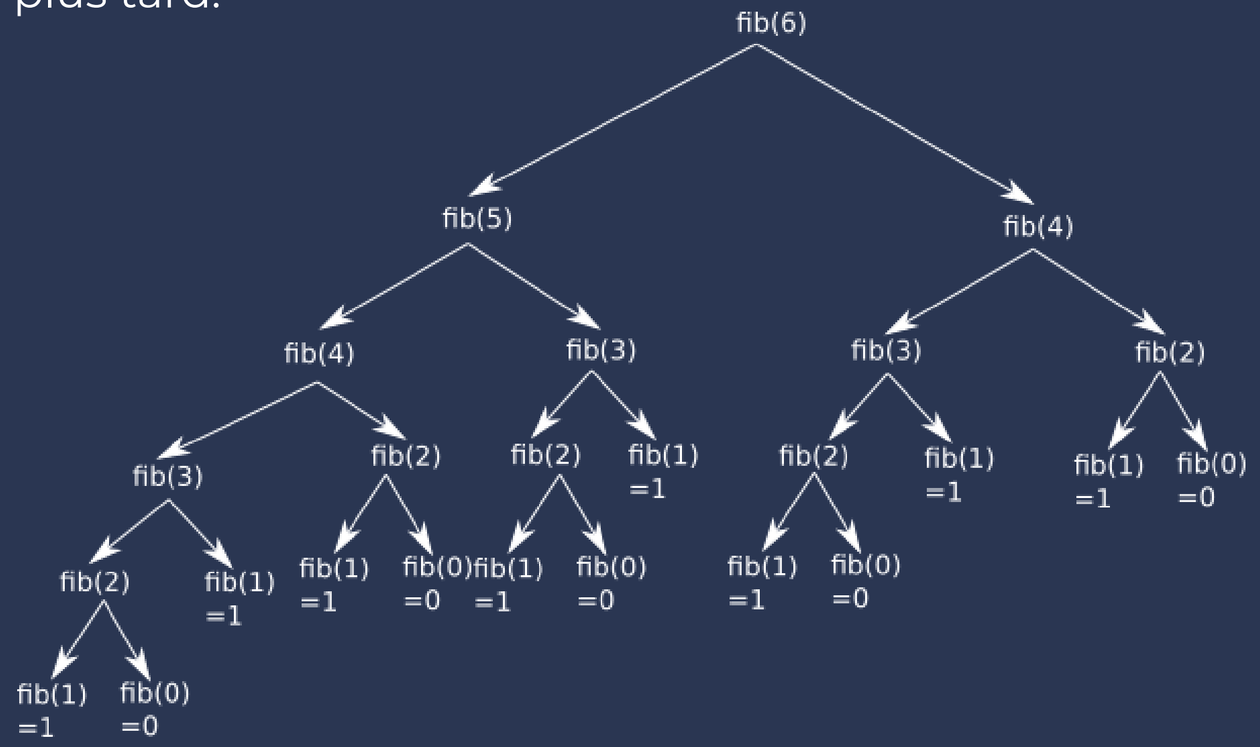
- **Exemple :**

Commençons par un exemple simple en utilisant la programmation dynamique 1D (à 1 dimension) : la suite de Fibonacci.
Rappelons son principe : fib(0) est égal à 0 et fib(1) est égal à 1. Pour calculer les termes suivants de la suite, il suffit d’additionner les deux termes précédents. Par exemple fib(2) est égal à 1 et fib(3) est égal à 2.

III - SOLUTION OPTIMISÉ

a. La programation dynamique

Comme vous pouvez le voir avec le schéma ci-dessous, on peut très vite se retrouver à calculer les mêmes termes plusieurs fois si on ne sauvegarde pas les précédents résultats. C'est là que la programmation dynamique intervient. Résoudre ce problème consisterai a partir de fib(0) et à calculer le termes un par un jusqu'à fib(6), tout en sauvegardant dans une liste chaque résultat, afin de les réutiliser plus tard.



III - SOLUTION OPTIMISÉ

a. La programation dynamique

- **Solution optimisé en pseudocode**

C’est donc ce principe de programmation dynamique que nous allons utiliser afin de développer une solution efficace pour maximiser les bénéfices de nos clients. Cette fois-ci la matrice sera en 2 dimensions et la fonction principale se décomposera en 3 parties :

- Création de la matrice
- Remplissage de la matrice
- Backtracking : Assigner la liste d’actions correspondant au bénéfices attribué à la dernière case

III - SOLUTION OPTIMISÉ

b. L'algorithme dynamique en pseudocode

- **Fonction**

La fonction prend en paramètre une liste d'action matéralisé sous formes d'instances de classe, ainsi que un budget maximum (précision : ce budget doit être entré sous forme d'unité, c'est à dire que 500€ devra plûtot être rentré sous forme de centimes)

- **Création de la matrice**

La matrice à 2 dimensions est un tableau : chaque colonne représente un budget, entre 0 et le budget max imposé, et chaque ligne représente une action (la premère ligne doit représenter aucune action). Ainsi chaque case représentera le bénéfice maximum que l'on peut tirer sous le budget imparti (la colonne) et la séléction d'actions imparti (l'action representé par la ligne + toutes celles qui ont été passé en revue (les lignes précédentes)).

```
FONCTION programmation_dynamique (liste_actions, budget_max) :

  #Création de la matrice vide
  matrice = []
  POUR i allant à taille(liste_actions) + 1 :
      ligne = []
      POUR j allant à budget_max + 1 :
          ligne.ajouter(0)
      matrice.ajouter(ligne)
```

III - SOLUTION OPTIMISÉ

b. L'algorithme dynamique en pseudocode

- **Remplissage de la matrice**

Une fois la matrice créée nous allons procéder de cette manière :
Nous allons pour chaque ligne, calculer le bénéfice optimale dans le budget imparti (colonne) en incluant ou non l'action (ligne).
Pour cela nous allons, pour chaque case, comparer le bénéfice optimale entre :

- L'action inclut (si coût rentre dans le budget) : bénéfice de l'action + bénéfice de la case qui exclut l'action (donc la ligne au dessus) et qui a pour budget le budget actuel - le coût de l'action
- L'action exclut : La ligne précédente

```
#Remplissage de la matrice
POUR action_index allant de 1 à taille(liste_actions) + 1 :
  action = liste_actions[action_index - 1]
  POUR budget allant de 1 à budget_max + 1 :
    SI action.coût <= budget :
      matrice[action_index][budget] = maximum(matrice[action_index - 1][budget],
      action.benefices + matrice[action_index - 1][budget-action.coût])
    SINON :
      matrice[action_index][budget] = matrice[action_index - 1][budget]
```

III - SOLUTION OPTIMISÉ

b. L’algorithme dynamique en pseudocode

- **Backtracking**

Faire du backtracking afin de retrouver les actions correspondantes au résultat final est moins coûteux en ressources que d’assigner une liste d’action à chaque case. Pour cela, une fois la matrice remplie, nous allons faire le chemin en sens inverse. Nous allons partir de la dernière case (donc séléction d’actions maximum et budget maximum) et comparer, dans le même budget, la ligne précédentes (donc en excluant l’action),

- Si le bénéfice est le même, cela signifie que l’action n’a pas été attribué.
- Si le bénéfice n’est pas le même, (donc correspond au bénéfice de l’action additionné au bénéfice de la case excluant l’action et ayant pour budget le budget actuel - le côut de l’action) alors l’action a été séléctionné.

Nous passons ensuite à la ligne d’au dessus, et si l’action à été séléctionné, nous passons alors également à la colonne ayant pour budget le coût de l’action soustrait au budget actuel.

```
#Retro engineering : Assigner une liste d'actions à la dernière case
liste_actions_optimale = []
budget_actuel = budget_max
index_action = taille(liste_actions)
TANT QUE budget_actuel > 0 ET index_action > 0 :
  action = liste_actions[index_action - 1]
  SI matrice[index_action][budget_actuel] == action.benefices + matrice[index_action-1]
  [budget_actuel - action.cout] :
    liste_actions_optimale.ajouter(action)
    budget_actuel -= action.cout
  index_action -= 1
```

III - SOLUTION OPTIMISÉ

b. L’algorithme dynamique en pseudocode

- **Cas limites**

1) Cette algorithme est cohérent dans la limite des conditions imposés, mais ne prend pas en compte la possibilité d’ajout de nouvelles contraintes plus complexes.

2) Il à également été conçu dans le cadre d’un problème algorithmique knapsack 0/1. C’est à dire avec l’impossibilité de fractionner les actions ou d’acheter une action plusieurs fois.

3) Enfin, malgré son efficacité, l’algorithme reste coûteux en ressources, si la data est trop énorme, le temps d’execution peut devenir long. Détaillons cela en comparant, et en utilisant la notation Big-O.

IV - COMPARAISON EFFICACITÉ ET PERFORMANCES AVEC BIG-O

a. Définition notation Big-O

- **Définition**

La notation Big O (ou grand O) est une notation mathématique utilisée en informatique pour décrire la complexité asymptotique d'un algorithme, c'est-à-dire la façon dont son temps d'exécution ou son utilisation mémoire évolue lorsque la taille de l'entrée augmente.

C'est une méthode qui permet de mesurer simplement la performance d'un algorithme. Sans prendre en compte les caractéristiques de la machine(langage, processeur, etc.), mais simplement en observant comment le nombre d'opérations ou la quantité de mémoire utilisée évolue.

IV - COMPARAISON EFFICACITÉ ET PERFORMANCES AVEC BIG-O

b. Brute-force vs version optimisé (programmation dynamique)

- **Comparaison de la complexité temporelle avec Big-O**

Force-brute

Pour n = taille de liste_actions

2^n

n

```
FONCTION brute_force(liste_actions):
  meilleure_combinaison = {combinaison_actions = [ ],
  cout_total = 0,benefice_total = 0}
  POUR combinaison_possible (liste_actions) :
    cout_total = 0
    benefice_total = 0
    POUR action DANS combinaison_possible:
      cout_total += action.cout
      benefice_total += action.benefice_euros
    SI cout_total <= 500 ET benefice_total > meilleure_combinaison
    ["benefice_total"] :
      meilleure_combinaison["combinaison_actions"] = combinaison_po
      ssible
      meilleure_combinaison["cout_total"] = cout_total
      meilleure_combinaison["benefice_total"] = benefice_total

  RETOURNE meilleure_combinaison
```

Complexité temporelle = O(n x 2^n)

Version optimisé (programmation dynamique)

Pour n = taille de liste_actions
Pour W = budget

n

W

n

W

n

```
FONCTION programmation_dynamique (liste_actions, budget_max) :

  #Création de la matrice vide
  matrice = []
  POUR i allant à taille(liste_actions) + 1 :
    ligne = []
    POUR j allant à budget_max + 1 :
      ligne.ajouter(0)
    matrice.ajouter(ligne)

  #Remplissage de la matrice
  POUR action_index allant de 1 à taille(liste_actions) + 1 :
    action = liste_actions[action_index - 1]
    POUR budget allant de 1 à budget_max + 1 :
      SI action.coût <= budget :
        matrice[action_index][budget] = maximum(matrice[acti
        on_index - 1][budget],action.benefices + matrice[act
        ion_index - 1][budget-action.coût])
      SINON :
        matrice[action_index][budget] = matrice[action_inde
        x - 1][budget]

  #Backtracking : Assigner une liste d'actions à la dernière case
  liste_actions_optimale = []
  budget_actuel = budget_max
  index_action = taille(liste_actions)
  TANT QUE budget_actuel > 0 ET index_action > 0 :
    action = liste_actions[index_action - 1]
    SI matrice[index_action][budget_actuel] == action.benefices +
    matrice[index_action-1]
    [budget_actuel - action.cout] :
      liste_actions_optimale.ajouter(action)
      budget_actuel -= action.cout
    index_action -= 1
```

Complexité temporelle de O(n x W + n x W + n)
soit ≃ O(n x W)

IV - COMPARAISON EFFICACITÉ ET PERFORMANCES AVEC BIG-O

b. Brute-force vs version optimisé (programmation dynamique)

- **Comparaison de la complexité temporelle avec Big-O**

La solution brute_force à une complexité temporelle exponentielle et donc trop coûteuse lorsque le nombre d'actions augmente significativement. La solution optimisé possède quant à elle un temps d'exécution polynomiale. Malgré le fait que l'on utilise majoritairement la notation Big-O pour mesurer la complexité temporelle, nous pouvons également l'utiliser afin d'analyser l'utilisation mémoire de la fonction.

- **Analyse de l'utilisation mémoire avec Big-O**

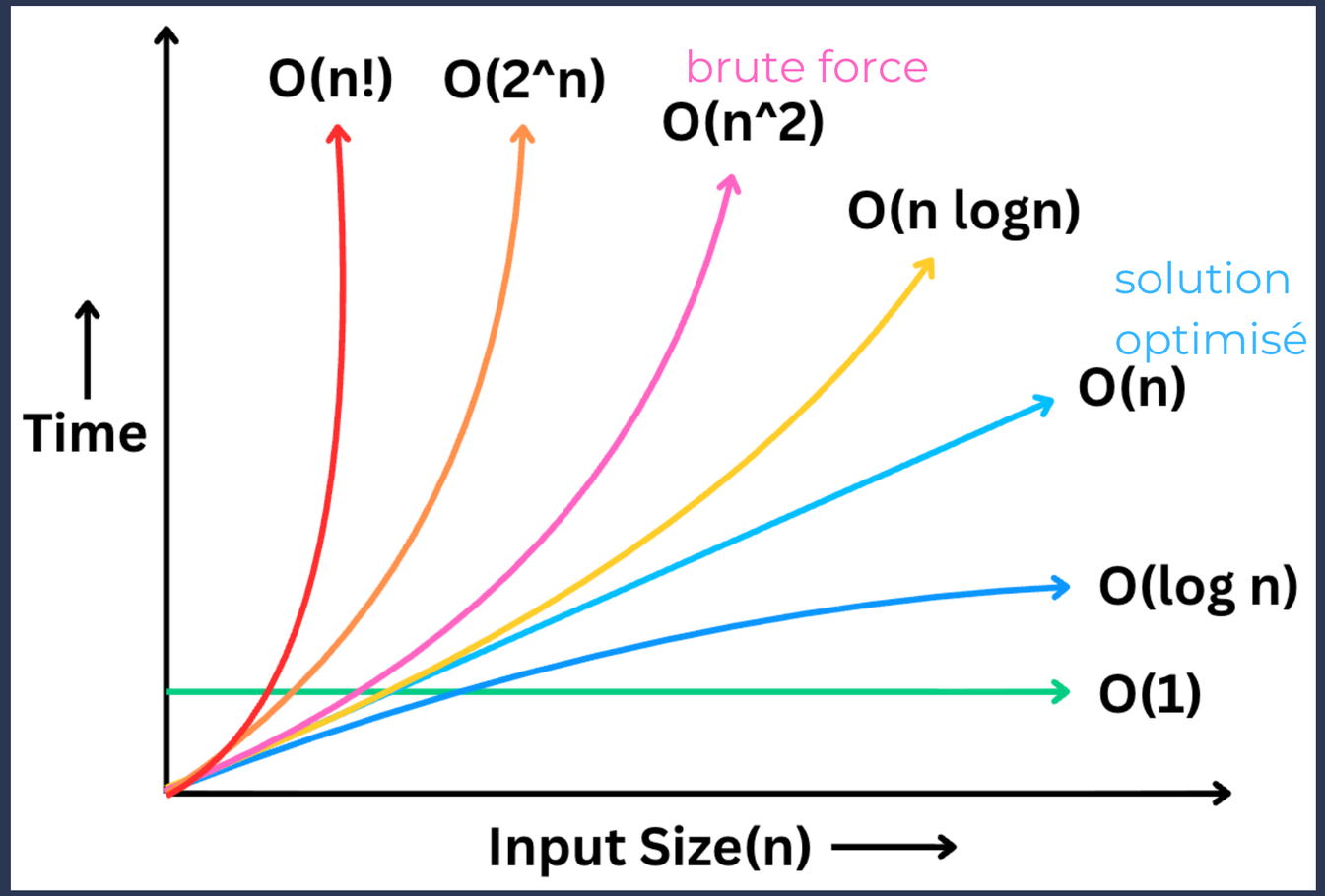
Lorsque nous calculons la complexité d'espace d'une fonction, nous ne prenons pas en compte l'entrée de mémoire, mais seulement la quantité de mémoire qui va devoir être créer selon les paramètres de la fonction.

La solution brute-force par exemple possède quelques variables mais qui reste constantes donc d'une complexité O(1), ainsi qu'une variable "combinaisons_possibles" qui est à chaque fois renouvelé et parfois stocké, d'une taille maximum de n, donc d'une complexité O(n). On peut donc noter que la **complexité totale de mémoire de la solution brute-force se note ≃ O(n)**
.

La version programmation dynamique quant à elle possède aussi quelque variable de taille constantes donc O(1), cependant elle stocke une matrice de taille n x W. Donc **la complexité de cette solution se note ≃ O(n x W).**

IV - COMPARAISON EFFICACITÉ ET PERFORMANCES AVEC BIG-O

b. Brute-force vs version optimisé (programmation dynamique)



V - ALGORITHME VS HUMAIN : COMPARAISON SUR DES DONNÉES ANTÉRIEURS

a. Présentation des données

- **Contexte**

Des données antérieurs m'ont été transmis par Sienna, ainsi que les choix d'investissments qui ont été fait manuellement à cette époque. C'est une bonne opportunité de montrer l'impact de l'automatisation et d'optimisation des choix d'invesstissment avec notre algorithme, mais également, au vu de la quantité de données, de tirer des informations intérressantes au travers d'un rapport d'exploration des données .

Voici un présentation des fichiers que nous allons analyser :

- dataset1 & 2 : Chaque fichier représente des centaines d'actions utilisé par l'entreprise il y'a quelques années. La variable price correspond au coût en euro de l'action représenté (elle ne peut être acheté qu'une fois et ne peut être fractionné) tandis que la variable profit représente le bénéfice en euro de l'action en question.

- sienna_decision_achat_dataset1 & 2 : Correspond à la sélection d'actions que Sienna à faite pour chaque dataset, pour un budget de 500€ maximum

- algorithme_decision_achat_dataset1 & 2 : Correspond à la sélection d'actions que l'algorithme à fait pour chaque dataset, pour un budget de 500€ maximum

V - ALGORITHME VS HUMAIN : COMPARAISON SUR DES DONNÉES ANTÉRIEURS

b. Rapports d’exploration

- **Rapport d'exploration dataset 1 & 2**

	Nombre d’actions	Nombre d’actions erronés (0 ou négatif)	Coût min	Coût max	Coût moy	Coût media n	Coût écart type	Bénefic e min	Bénefice max	Bénefice moy	Bénefice median	Bénefice ecart-type
DATASET 1	1001	44	0.01€	498.76€	25.54€	25.62€	19.37€	0.02%	39.98%	20.23%	19.82%	11.48%
DATASET 2	1000	459	0.12€	51.46€	25.61€	26.12€	10.50€	0.15%	39.97%	19.57%	19.71%	12.20%

V - ALGORITHME VS HUMAIN : COMPARAISON SUR DES DONNÉES ANTÉRIEURS

b. Rapports d'exploration

- **Rapport d'exploration choix de Sienna vs choix de l'algorithme**

DATASET1	Sienna	Algorithme
Temps d'execution	?	8 secondes
Coût total	498,76€	499,96€
Bénéfice net	196,61€	198,55€

DATASET2	Sienna	Algorithme
Temps d'execution	?	5 secondes
Coût total	489,24€	499,92€
Bénéfice net	193,78€	197,96€

V - ALGORITHME VS HUMAIN : COMPARAISON SUR DES DONNÉES ANTÉRIEURS

b. Rapports d’exploration

- **Rapport d'exploration choix de Sienna vs choix de l’algorithme**

Dataset 1 & 2 :

Nous pouvons observer que le temps d’execution de l’algorithme Python était de seulement quelques secondes pour un nombre conséquent d’action (1000), comparé à surement plusieurs heures pour Sienna. Mais aussi que étant donné la complexité temporelle de la solution optimisé, le temps d’execution ne risque pas d’etre exponentiel au fur et a mesure que la data augmente, et peut donc être prédictible.

Ajoutons également que plus de budget à été investi par l’algorithme et que par la suite plus de bénéfice a pu, dans les deux dataset, être produit.

On peut en conclure que ce n’est pas seulement plus optimal en termes de rapidité d’execution d’automatiser nos programmes d’investissements, c’est surtout plus optimal en termes de performance pure du produit que nous délivrons.